Introduction

- We are interested in formalizing UR learning as a specific kind of function decomposition.
- Hua and Jardine (2021) formalize the UR learning problem as a decomposition of two functions:

  a UR assignment function f and a phonology function g.

  The learner gets data from $g \circ f$ and must return representations of f and g.

Introduction

- We are interested in formalizing UR learning as a specific kind of function decomposition.
- Hua and Jardine (2021) formalize the UR learning problem as a decomposition of two functions:

  a UR assignment function f and a phonology function g.

  The learner gets data from $g \circ f$ and must return representations of f and g.

Introduction

- We are interested in formalizing UR learning as a specific kind of function decomposition.
- Hua and Jardine (2021) formalize the UR learning problem as a decomposition of two functions:

  a UR assignment function f and a phonology function g.

  The learner gets data from $g \circ f$ and must return representations of f and g.

UR Learning as Function Decomposition

- $\mathcal{M}$ = abstract morpheme labels e.g. {CAT, RUN, PL, PAST, . . . }
- Σ = input alphabet of segments e.g. {k, ɪ, ʃ, æ, ŋ, . . . }
- Γ = output alphabet of segments e.g. {k, ɪ, ʃ, æ, ŋ, . . . }

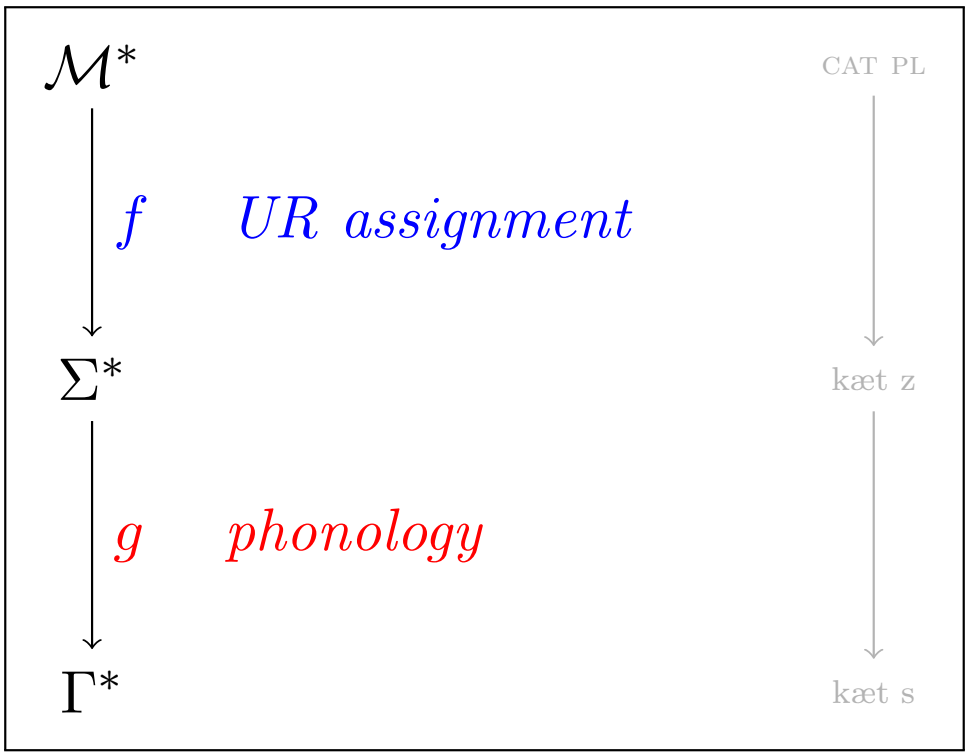


Figure: Schematization of Morpho-Phonological Production

What are our Restrictions?

- These can't just be any functions! (*nightmare*)

  Different function classes have different learnability properties.

  If f, g could be *any kind of function*, we wouldn't get many satisfying results...

- So Hua and Jardine (2021) consider very specific classes of functions.

What are our Restrictions?

- These can't just be any functions! (*nightmare*)

  Different function classes have different learnability properties.

  If f, g could be *any kind of function*, we wouldn't get many satisfying results...

- So Hua and Jardine (2021) consider very specific classes of functions.

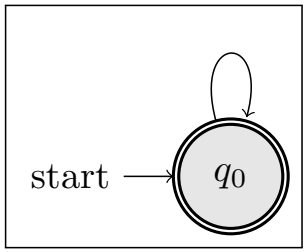
What Kind of Functions?

- To keep things manageable, Hua and Jardine (2021) enforce the following restrictions:
  - The UR assignment function f is a letter-to-string homomorphism, always a *one-state machine* T_f:

  - The phonology function g is a Simplex ISL_2 function always with a machine T_g with the *form below*:

What Kind of Functions?

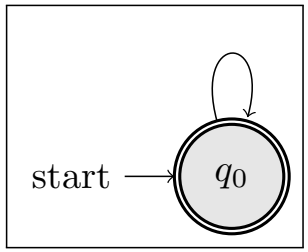
- To keep things manageable, Hua and Jardine (2021) enforce the following restrictions:
  - The UR assignment function f is a letter-to-string homomorphism, always a *one-state machine* T_f:



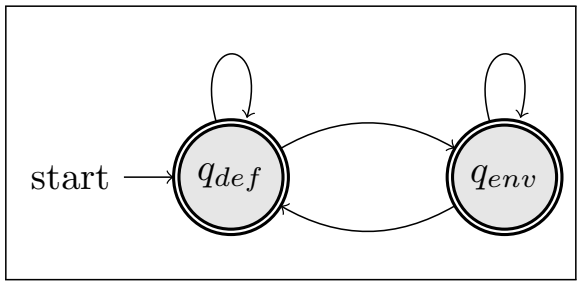
  - The phonology function g is a Simplex ISL_2 function always with a machine T_g with the *form below*:

What Kind of Functions?

- To keep things manageable, Hua and Jardine (2021) enforce the following restrictions:
  - The UR assignment function f is a letter-to-string homomorphism, always a *one-state machine* T_f:



  - The phonology function g is a Simplex ISL_2 function always with a machine T_g with the *form below*:



Extending These Restrictions

- "Simplex" simply enforces that only a single change can be made in a single environment at a time.

- This makes the problem far more approachable, but it systematically misses certain generalizations.

  For example, it cannot learn paradigms for which phonology function g realizes *an iterative pattern*.

Main Contribution

We aim to expand the empirical coverage of this learner by incorporating tier information.

Extending These Restrictions

- "Simplex" simply enforces that only a single change can be made in a single environment at a time.
- This makes the problem far more approachable, but it systematically misses certain generalizations.

  For example, it cannot learn paradigms for which phonology function g realizes *an iterative pattern*.

Main Contribution

We aim to expand the empirical coverage of this learner by incorporating tier information.

Extending These Restrictions

- "Simplex" simply enforces that only a single change can be made in a single environment at a time.
- This makes the problem far more approachable, but it systematically misses certain generalizations.

  For example, it cannot learn paradigms for which phonology function g realizes *an iterative pattern*.

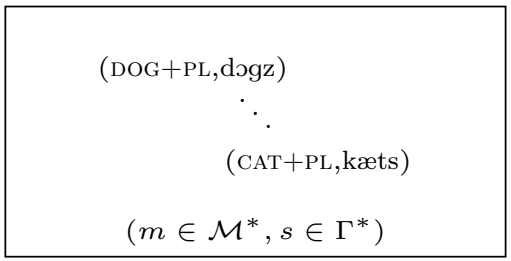
Main Contribution

We aim to expand the empirical coverage of this learner by incorporating tier information.

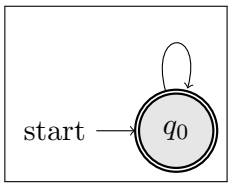
Visualization of Learning Problem

I'm a morpho-phonological learner named SI_2DLA.

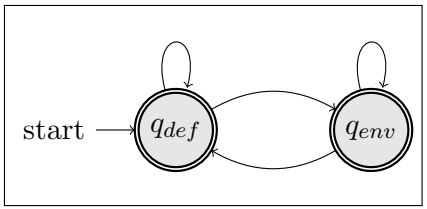
You hand me a finite sample of (in, out) pairs, and my goal is to give you back two nice representations:



Morphology: T_f



Phonology: T_g



SI_2DLA Procedure: Broad Strokes

1. Start with sample D from $g \circ f$ and use OSTIA (Oncina et al., 1993) to construct a two-state SFST $T_{g \circ f}$ consistent with D.
2. For both states in $T_{g \circ f}$, read the output 1-suffixes: set the state with
   - *more* output suffixes to q_{def} in T_g
   - *a single* output suffix to q_{env} in T_g
3. Set input suffixes for T_g's states as the output suffixes from the last step, and then remove common element, using this to build T_g's transition function.
4. Modify $T_{g \circ f}$ by deleting outgoing transitions from the state corresponding to q_{env} and merge the two states, yielding the one-state machine T_f.

SI_2DLA Procedure: Broad Strokes

1. Start with sample D from $g \circ f$ and use OSTIA (Oncina et al., 1993) to construct a two-state SFST $T_{g \circ f}$ consistent with D.
2. For both states in $T_{g \circ f}$, read the output 1-suffixes: set the state with
   - *more* output suffixes to q_{def} in T_g
   - *a single* output suffix to q_{env} in T_g
3. Set input suffixes for T_g's states as the output suffixes from the last step, and then remove common element, using this to build T_g's transition function.
4. Modify $T_{g \circ f}$ by deleting outgoing transitions from the state corresponding to q_{env} and merge the two states, yielding the one-state machine T_f.

SI_2DLA Procedure: Broad Strokes

1. Start with sample D from $g \circ f$ and use OSTIA (Oncina et al., 1993) to construct a two-state SFST $T_{g \circ f}$ consistent with D.

2. For both states in $T_{g \circ f}$, read the output 1-suffixes: set the state with
   - *more* output suffixes to q_{def} in T_g
   - *a single* output suffix to q_{env} in T_g

3. Set input suffixes for T_g's states as the output suffixes from the last step, and then remove common element, using this to build T_g's transition function.

4. Modify $T_{g \circ f}$ by deleting outgoing transitions from the state corresponding to q_{env} and merge the two states, yielding the one-state machine T_f.

SI_2DLA Procedure: Broad Strokes

1. Start with sample D from $g \circ f$ and use OSTIA (Oncina et al., 1993) to construct a two-state SFST $T_{g \circ f}$ consistent with D.

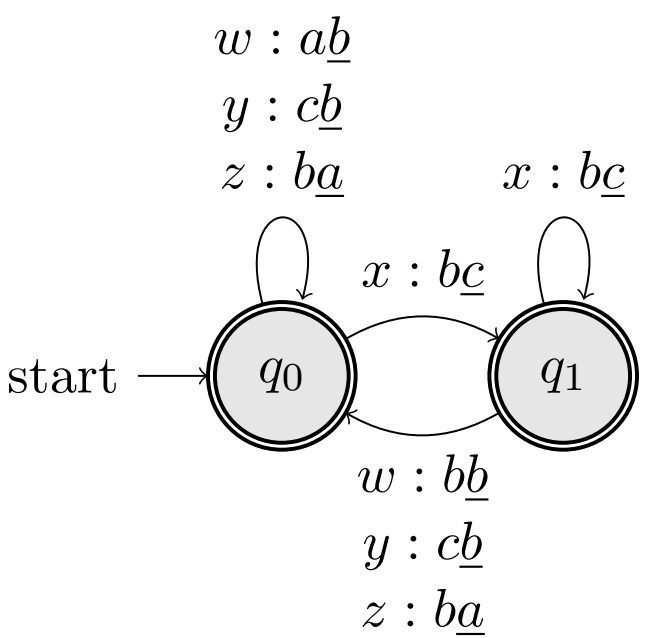
2. For both states in $T_{g \circ f}$, read the output 1-suffixes: set the state with
   - *more* output suffixes to q_{def} in T_g
   - *a single* output suffix to q_{env} in T_g

3. Set input suffixes for T_g's states as the output suffixes from the last step, and then remove common element, using this to build T_g's transition function.

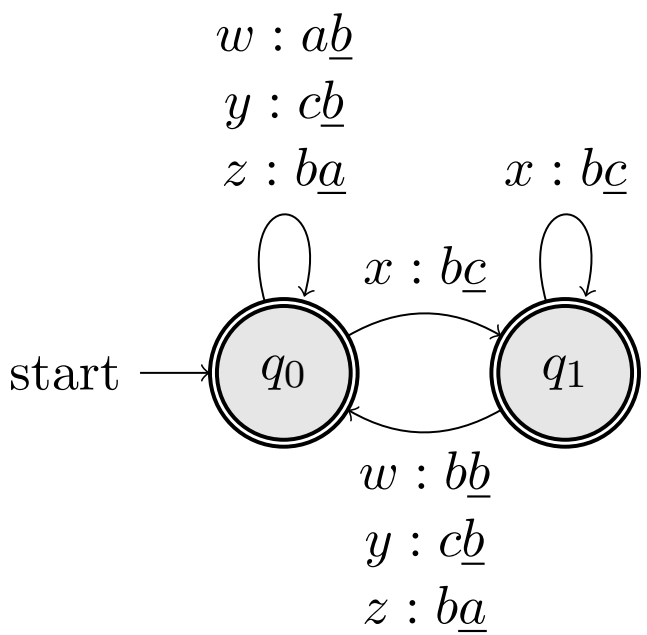
4. Modify $T_{g \circ f}$ by deleting outgoing transitions from the state corresponding to q_{env} and merge the two states, yielding the one-state machine T_f.

Example Target Functions

Example

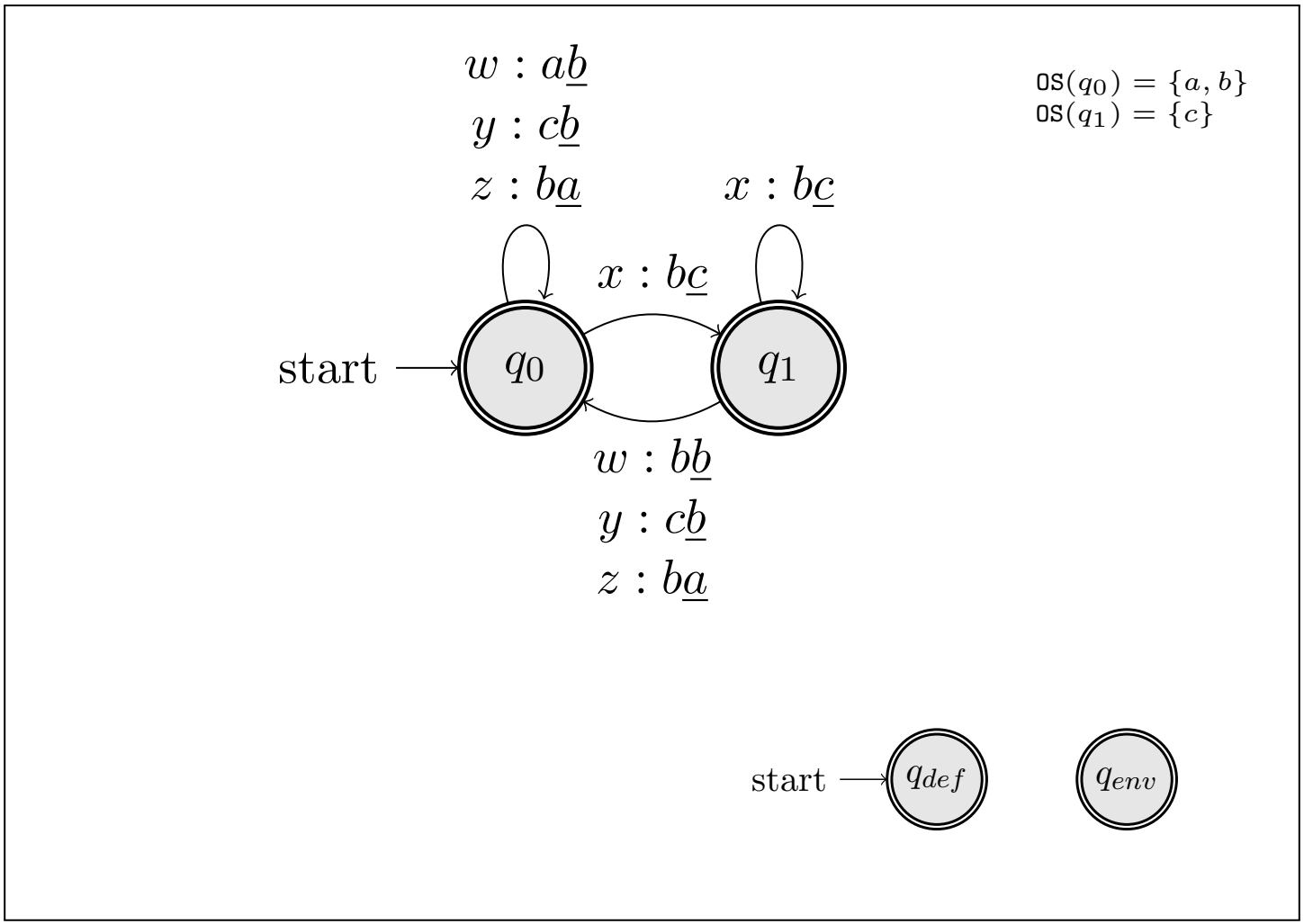

$w : a\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$x : b\underline{c}$
$x : b\underline{c}$
start
q_0
q_1
$w : b\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$

Example

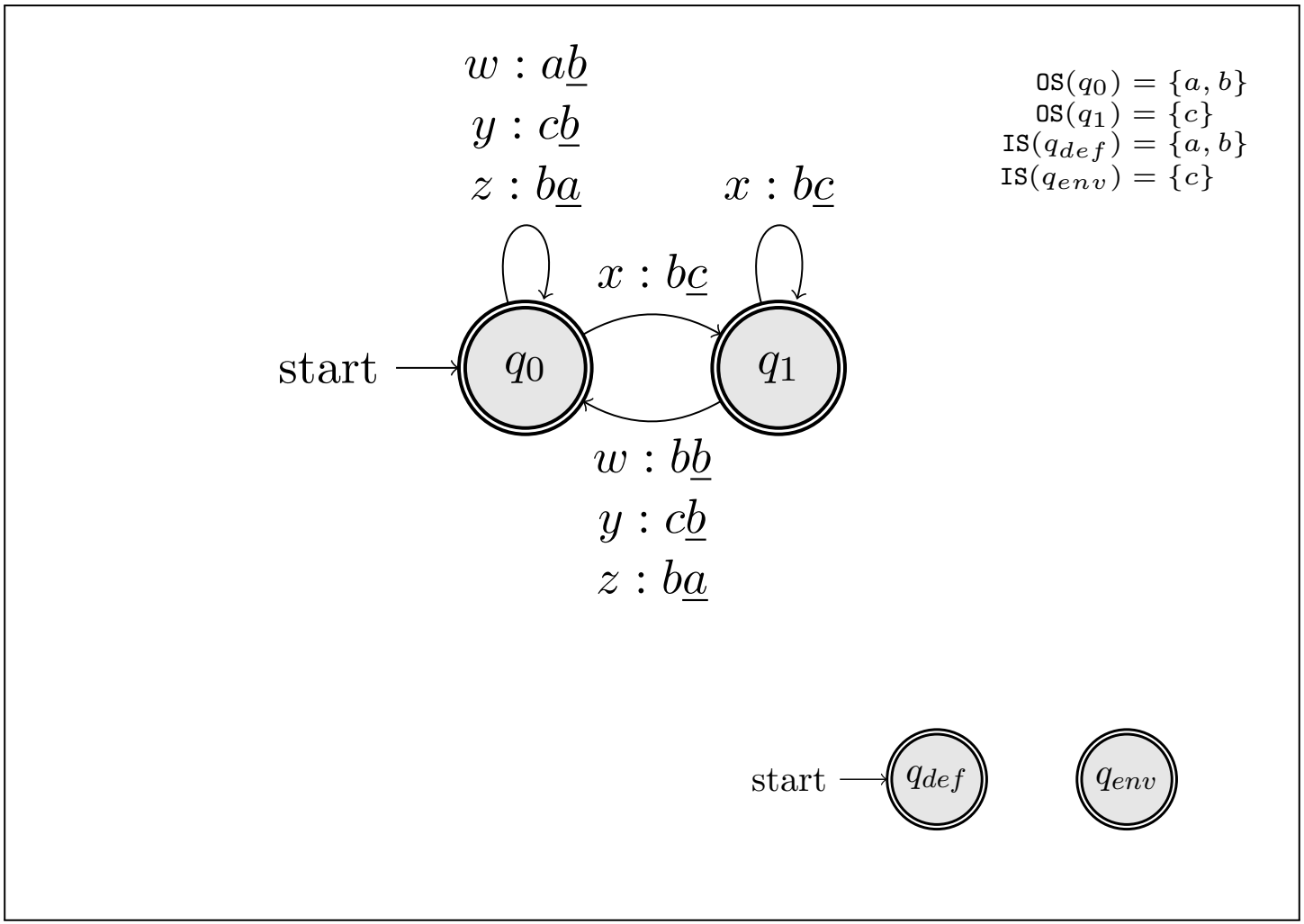


$\texttt{OS}(q_0) = \{a, b\}$
$\texttt{OS}(q_1) = \{c\}$

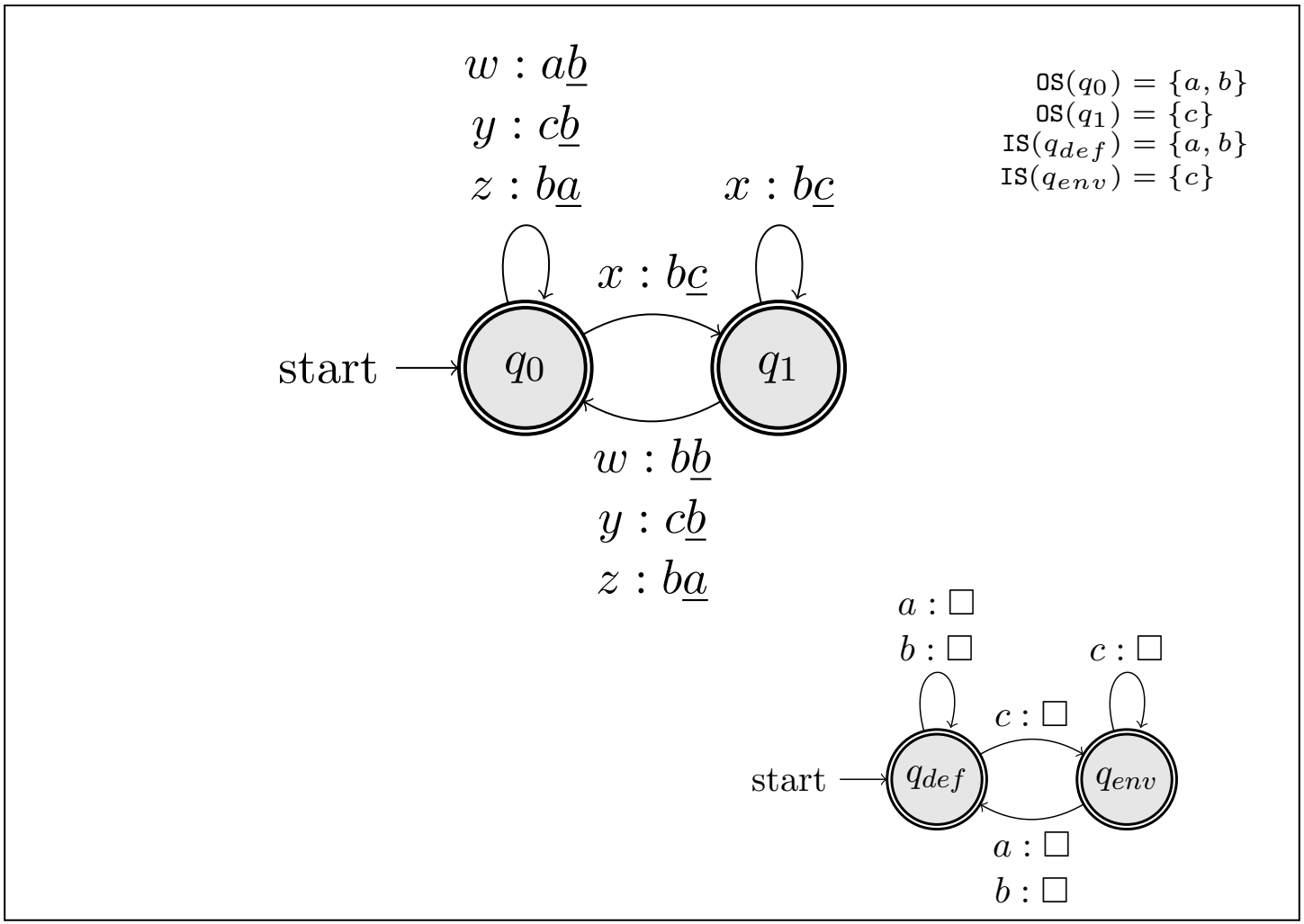
Example


$w : a\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$x : b\underline{c}$
$\texttt{OS}(q_0) = \{a, b\}$
$\texttt{OS}(q_1) = \{c\}$
$x : b\underline{c}$
start
q_0
q_1
$w : b\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
start
q_{def}
q_{env}

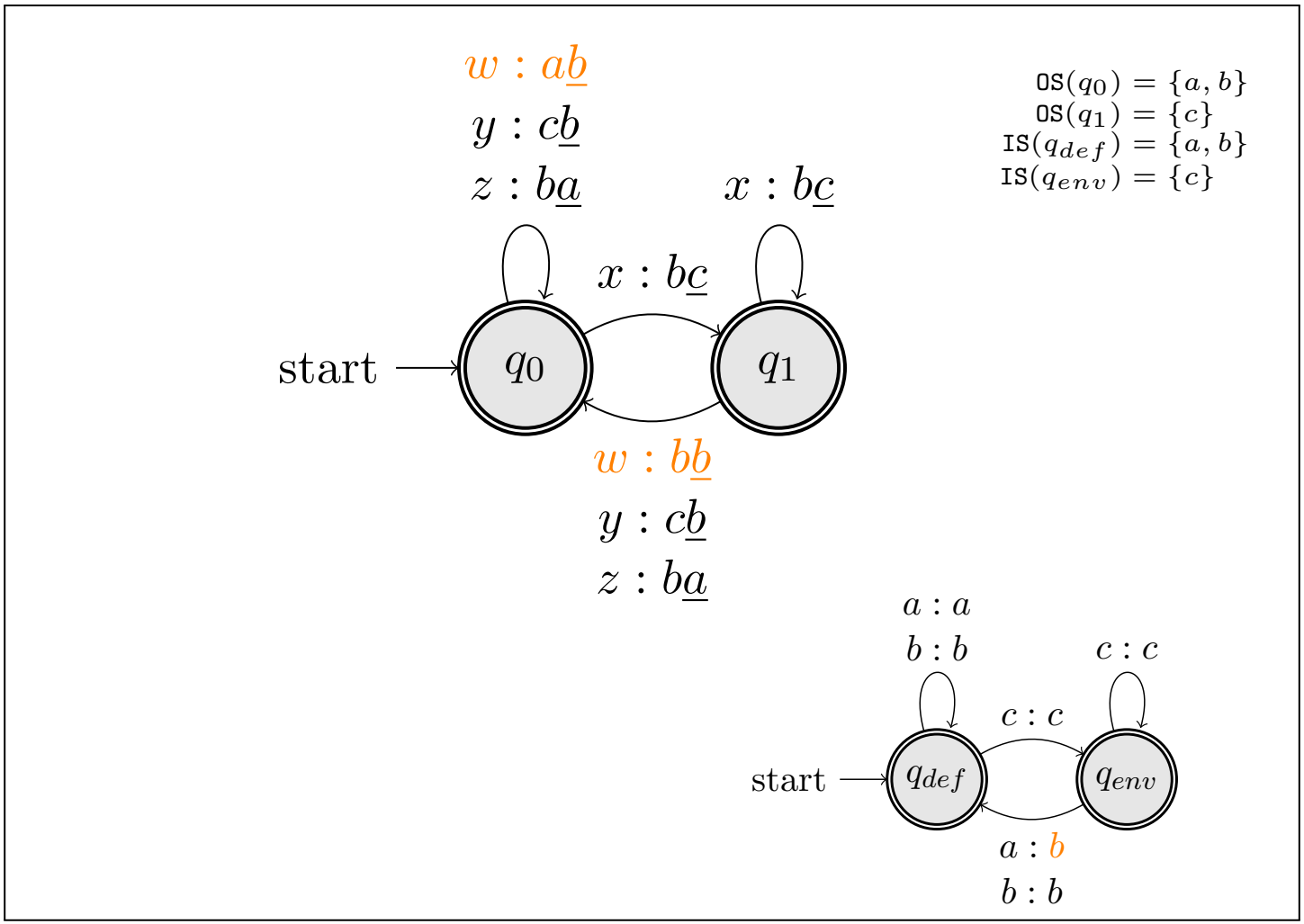
Example


$w : a\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$x : b\underline{c}$
$\texttt{OS}(q_0) = \{a, b\}$
$\texttt{OS}(q_1) = \{c\}$
$\texttt{IS}(q_{def}) = \{a, b\}$
$\texttt{IS}(q_{env}) = \{c\}$
$x : b\underline{c}$
start
q_0
q_1
$w : b\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
start
q_{def}
q_{env}

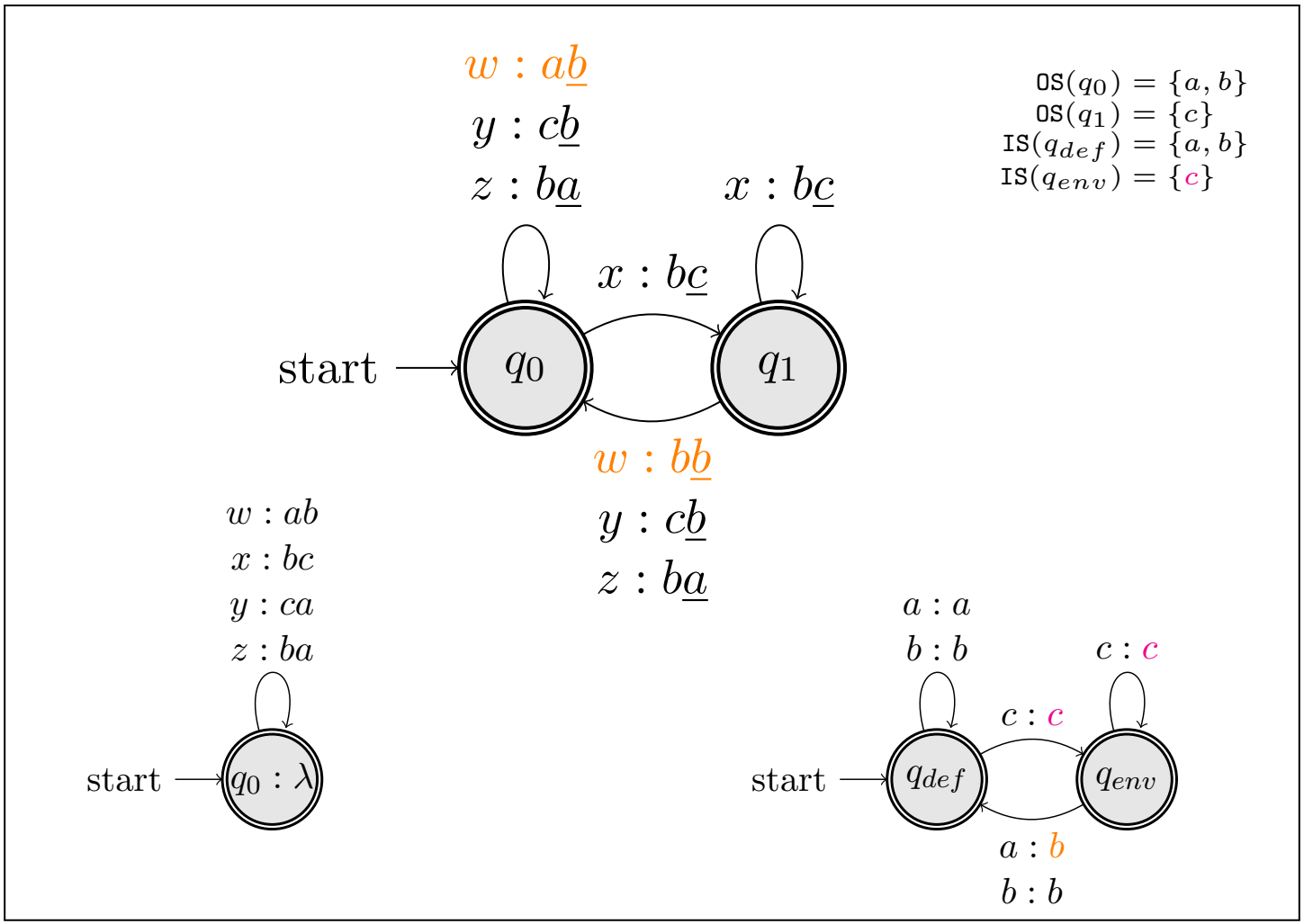
Example


$w : a\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$x : b\underline{c}$
$\texttt{OS}(q_0) = \{a, b\}$
$\texttt{OS}(q_1) = \{c\}$
$\texttt{IS}(q_{def}) = \{a, b\}$
$\texttt{IS}(q_{env}) = \{c\}$
$x : b\underline{c}$
start
q_0
q_1
$w : b\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$a : \square$
$b : \square$
$c : \square$
$c : \square$
start
q_{def}
q_{env}
$a : \square$
$b : \square$

Example


$w : a\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$x : b\underline{c}$
$\texttt{OS}(q_0) = \{a, b\}$
$\texttt{OS}(q_1) = \{c\}$
$\texttt{IS}(q_{def}) = \{a, b\}$
$\texttt{IS}(q_{env}) = \{c\}$
$x : b\underline{c}$
start
q_0
q_1
$w : b\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$a : a$
$b : b$
$c : c$
$c : c$
start
q_{def}
q_{env}
$a : b$
$b : b$

Example


$w : a\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$x : b\underline{c}$
$\mathtt{OS}(q_0) = \{a, b\}$
$\mathtt{OS}(q_1) = \{c\}$
$\mathtt{IS}(q_{def}) = \{a, b\}$
$\mathtt{IS}(q_{env}) = \{c\}$
$x : b\underline{c}$
start
q_0
q_1
$w : b\underline{b}$
$y : c\underline{b}$
$z : b\underline{a}$
$w : ab$
$x : bc$
$y : ca$
$z : ba$
start
$q_0 : \lambda$
$a : a$
$b : b$
$c : c$
$c : c$
start
q_{def}
q_{env}
$a : b$
$b : b$

Comparing ISL and TISL Machines

Note that these two machines are exactly the same, except for a single transition (highlighted in purple).

ISL

each state = $k-1$ input suffix

TISL

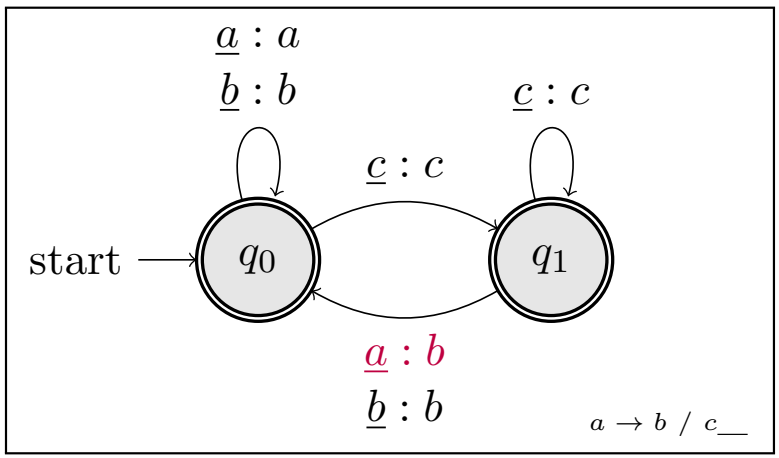
each state = $k-1$ input suffix *on a tier*

In the TISL machine, the symbol a loops on both states. If we ignore a, the machine is ISL (Lambert and Heinz, 2024)! (*hmm...*)

Comparing ISL and TISL Machines

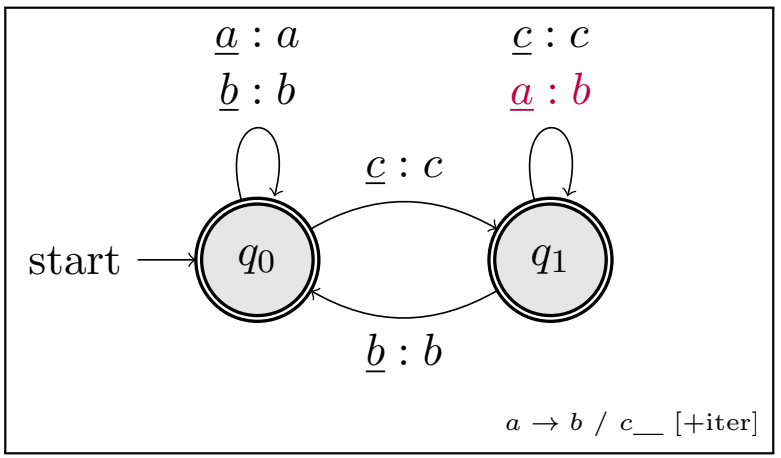
Note that these two machines are exactly the same, except for a single transition (highlighted in purple).

ISL



each state = $k - 1$ input suffix

TISL



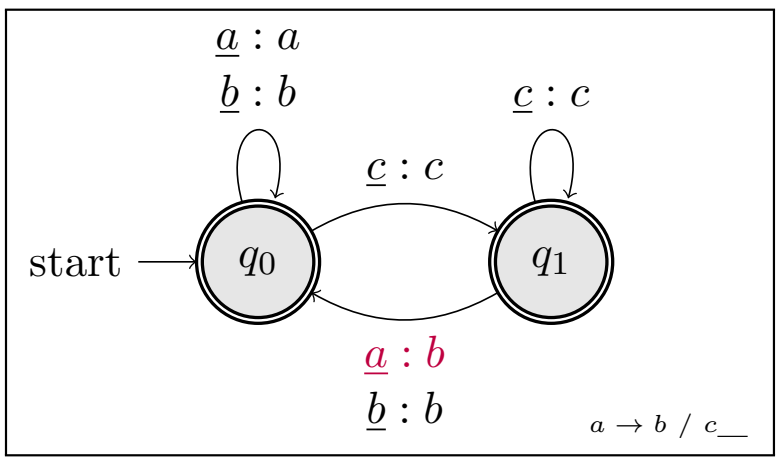
each state = $k - 1$ input suffix *on a tier*

In the TISL machine, the symbol a loops on both states. If we ignore a, the machine is ISL (Lambert and Heinz, 2024)! (*hmm...*)

Comparing ISL and TISL Machines

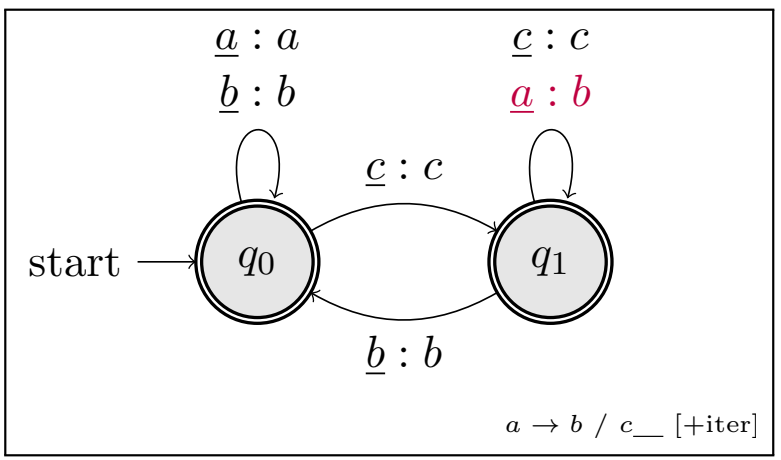
Note that these two machines are exactly the same, except for a single transition (highlighted in purple).

ISL



each state = $k-1$ input suffix

TISL



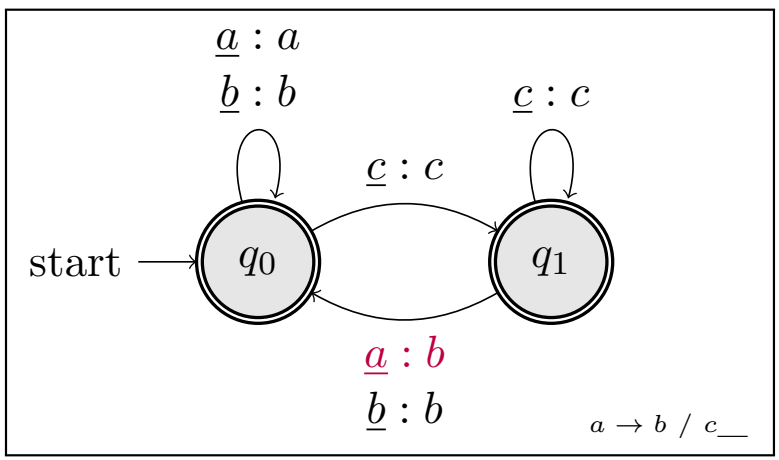
each state = $k-1$ input suffix *on a tier*

In the TISL machine, the symbol a loops on both states. If we ignore a, the machine is ISL (Lambert and Heinz, 2024)! (*hmm...*)

Comparing ISL and TISL Machines

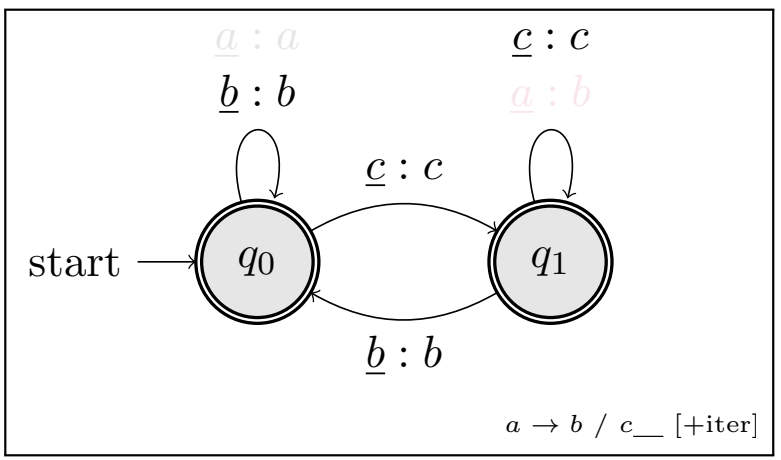
Note that these two machines are exactly the same, except for a single transition (highlighted in purple).

ISL



each state = $k-1$ input suffix

TISL



each state = $k-1$ input suffix *on a tier*

In the TISL machine, the symbol a loops on both states. If we ignore a, the machine is ISL (Lambert and Heinz, 2024)! (*hmm...*)

Underlying Representation Learning with Tiers

We are going to make broadly similar assumptions to those in Hua and Jardine (2021). Here are our assumptions about our targets:

- T_g is a simplex ISL_2 transducer.
- T_f is a homomorphism.
- T_g is length preserving (our only added assumption).

We're going to use an extended example to illustrate how the algorithm works. For the sake of time, we are also mainly just going to illustrate one method for finding neutral symbols.

Underlying Representation Learning with Tiers

We are going to make broadly similar assumptions to those in Hua and Jardine (2021). Here are our assumptions about our targets:

- T_g is a simplex ISL_2 transducer.
- T_f is a homomorphism.
- T_g is length preserving (our only added assumption).

We're going to use an extended example to illustrate how the algorithm works. For the sake of time, we are also mainly just going to illustrate one method for finding neutral symbols.

Underlying Representation Learning with Tiers

We are going to make broadly similar assumptions to those in Hua and Jardine (2021). Here are our assumptions about our targets:

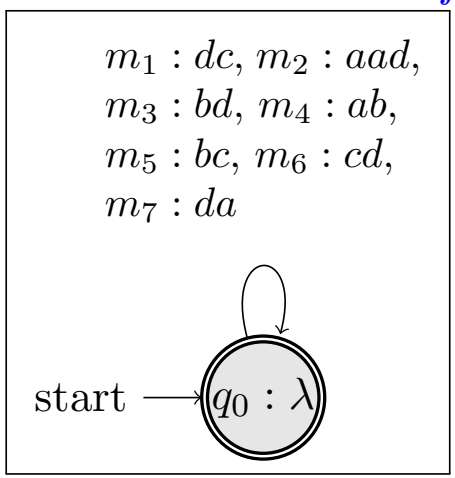
- T_g is a simplex ISL_2 transducer.
- T_f is a homomorphism.
- T_g is length preserving (our only added assumption).

We're going to use an extended example to illustrate how the algorithm works. For the sake of time, we are also mainly just going to illustrate one method for finding neutral symbols.

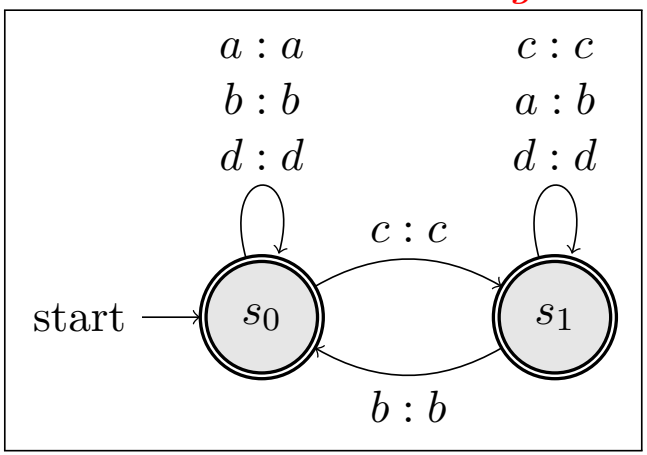
Example Targets

We are going to use some example targets functions:

Morphology: $T_{\hat{f}}$



Phonology: $T_{\hat{g}}$



$\mathcal{M} = \{m_1, m_2, m_3, m_4, m_5, m_6, m_7\}$

$\Gamma = \{a, b, c, d\} = \Delta$

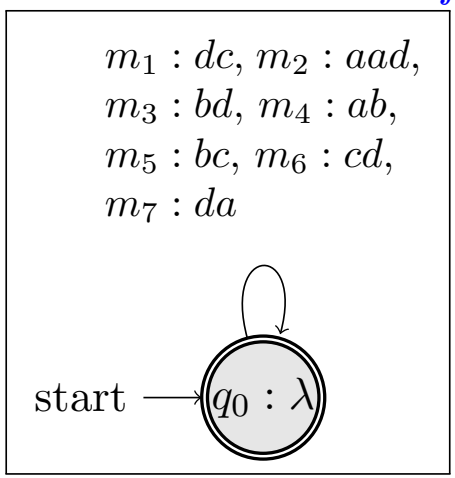
$\tau = \{c, b\}$

These are essentially just iterative versions of the target functions we learned with SI_2DLA.

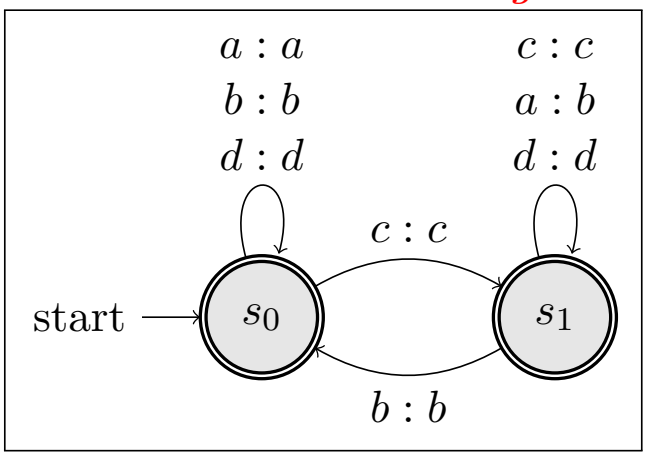
Example Targets

We are going to use some example targets functions:

Morphology: $T_{\hat{f}}$



Phonology: $T_{\hat{g}}$



$\mathcal{M} = \{m_1, m_2, m_3, m_4, m_5, m_6, m_7\}$

$\Gamma = \{a, b, c, d\} = \Delta$

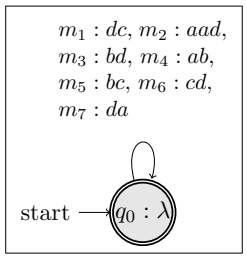
$\tau = \{c, b\}$

These are essentially just iterative versions of the target functions we learned with $\mathrm{SI}_2\mathrm{DLA}$.

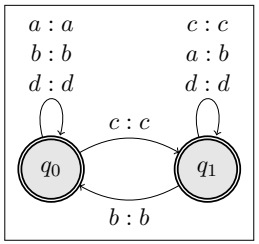
Composition

We compose these targets:

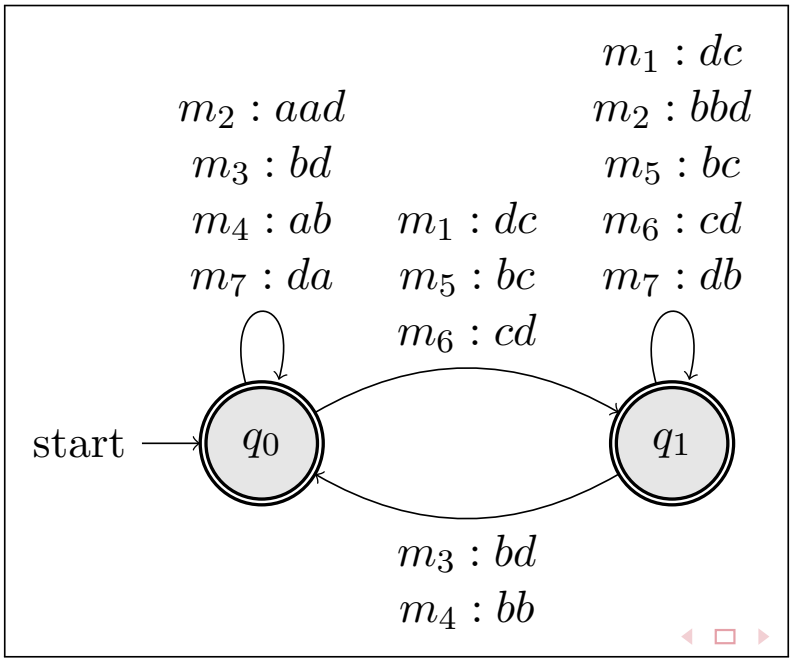
Morphology: T_f



Phonology: T_g



Morphophonology: $T_g \circ T_f$



Algorithm Outline

1. Just like in SI_2DLA, we start with a sample D from $\hat{g} \circ \hat{f} = \hat{h}$ and use OSTIA (Oncina et al. (1993)) to construct a two-state SFST $T_h = T_{\hat{h}} = T_{\hat{g} \circ \hat{f}}$.
2. Just like in SI_2DLA, again, for both states in $T_{\hat{h}}$, get the output suffixes with respect to each state and set:
   - the state with more output suffixes to q_{def}
   - the state with fewer output suffixes to q_{env}
3. Find the trigger from these output suffixes (similar though distinct to SI_2DLA).
4. Find the neutral symbols:
   - If there are environments where a symbol comes to the left of the target, it is neutral.
   - If there are environments where a symbol comes to the right of the trigger, it is neutral.
   - Everything else is on the tier.

Algorithm Outline

1. Just like in SI_2DLA, we start with a sample D from $\hat{g} \circ \hat{f} = \hat{h}$ and use OSTIA (Oncina et al. (1993)) to construct a two-state SFST $T_h = T_{\hat{h}} = T_{\hat{g} \circ \hat{f}}$.
2. Just like in SI_2DLA, again, for both states in $T_{\hat{h}}$, get the output suffixes with respect to each state and set:
   - the state with more output suffixes to q_{def}
   - the state with fewer output suffixes to q_{env}
3. Find the trigger from these output suffixes (similar though distinct to SI_2DLA).
4. Find the neutral symbols:
   - If there are environments where a symbol comes to the left of the target, it is neutral.
   - If there are environments where a symbol comes to the right of the trigger, it is neutral.
   - Everything else is on the tier.

Algorithm Outline

1. Just like in SI_2DLA, we start with a sample D from $\hat{g} \circ \hat{f} = \hat{h}$ and use OSTIA (Oncina et al. (1993)) to construct a two-state SFST $T_h = T_{\hat{h}} = T_{\hat{g} \circ \hat{f}}$.
2. Just like in SI_2DLA, again, for both states in $T_{\hat{h}}$, get the output suffixes with respect to each state and set:
   - the state with more output suffixes to q_{def}
   - the state with fewer output suffixes to q_{env}
3. Find the trigger from these output suffixes (similar though distinct to SI_2DLA).
4. Find the neutral symbols:
   - If there are environments where a symbol comes to the left of the target, it is neutral.
   - If there are environments where a symbol comes to the right of the trigger, it is neutral.
   - Everything else is on the tier.

Algorithm Outline

1. Just like in $\mathrm{SI}_2\mathrm{DLA}$, we start with a sample D from $\hat{g} \circ \hat{f} = \hat{h}$ and use OSTIA (Oncina et al. (1993)) to construct a two-state SFST $T_h = T_{\hat{h}} = T_{\hat{g} \circ \hat{f}}$.
2. Just like in $\mathrm{SI}_2\mathrm{DLA}$, again, for both states in $T_{\hat{h}}$, get the output suffixes with respect to each state and set:
   - the state with more output suffixes to q_{def}
   - the state with fewer output suffixes to q_{env}
3. Find the trigger from these output suffixes (similar though distinct to $\mathrm{SI}_2\mathrm{DLA}$).
4. Find the neutral symbols:
   - If there are environments where a symbol comes to the left of the target, it is neutral.
   - If there are environments where a symbol comes to the right of the trigger, it is neutral.
   - Everything else is on the tier.

Algorithm Outline

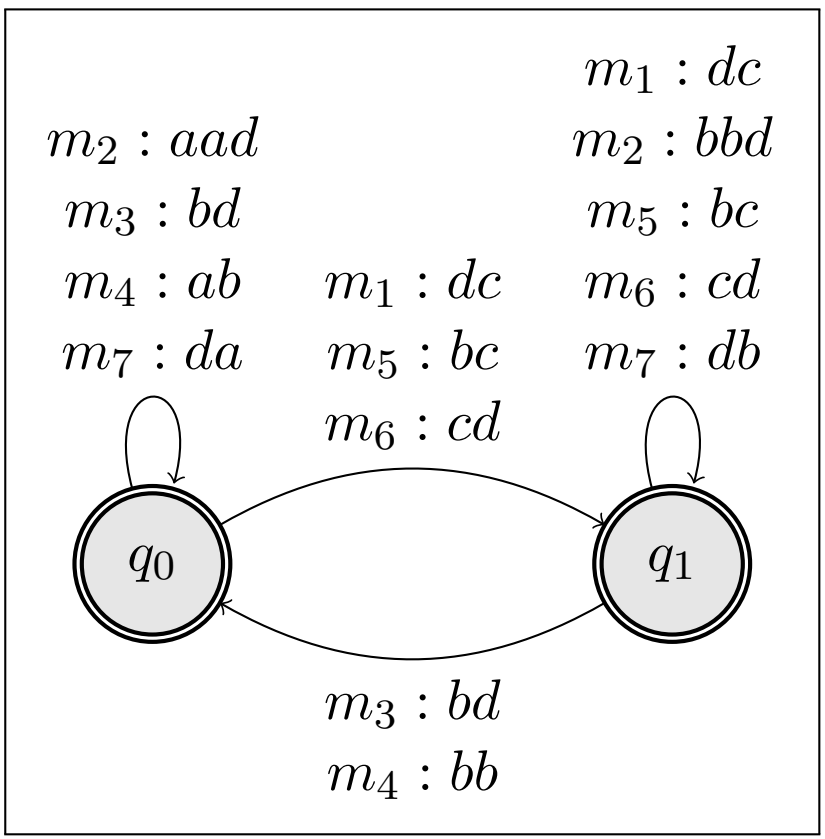
1. Just like in $\mathrm{SI}_2\mathrm{DLA}$, we start with a sample D from $\hat{g} \circ \hat{f} = \hat{h}$ and use OSTIA (Oncina et al. (1993)) to construct a two-state SFST $T_h = T_{\hat{h}} = T_{\hat{g} \circ \hat{f}}$.
2. Just like in $\mathrm{SI}_2\mathrm{DLA}$, again, for both states in $T_{\hat{h}}$, get the output suffixes with respect to each state and set:
   - the state with more output suffixes to q_{def}
   - the state with fewer output suffixes to q_{env}
3. Find the trigger from these output suffixes (similar though distinct to $\mathrm{SI}_2\mathrm{DLA}$).
4. Find the neutral symbols:
   - If there are environments where a symbol comes to the left of the target, it is neutral.
   - If there are environments where a symbol comes to the right of the trigger, it is neutral.
   - Everything else is on the tier.

Algorithm Outline

1. Just like in SI_2DLA, we start with a sample D from $\hat{g} \circ \hat{f} = \hat{h}$ and use OSTIA (Oncina et al. (1993)) to construct a two-state SFST $T_h = T_{\hat{h}} = T_{\hat{g} \circ \hat{f}}$.
2. Just like in SI_2DLA, again, for both states in $T_{\hat{h}}$, get the output suffixes with respect to each state and set:
   - the state with more output suffixes to q_{def}
   - the state with fewer output suffixes to q_{env}
3. Find the trigger from these output suffixes (similar though distinct to SI_2DLA).
4. Find the neutral symbols:
   - If there are environments where a symbol comes to the left of the target, it is neutral.
   - If there are environments where a symbol comes to the right of the trigger, it is neutral.
   - Everything else is on the tier.

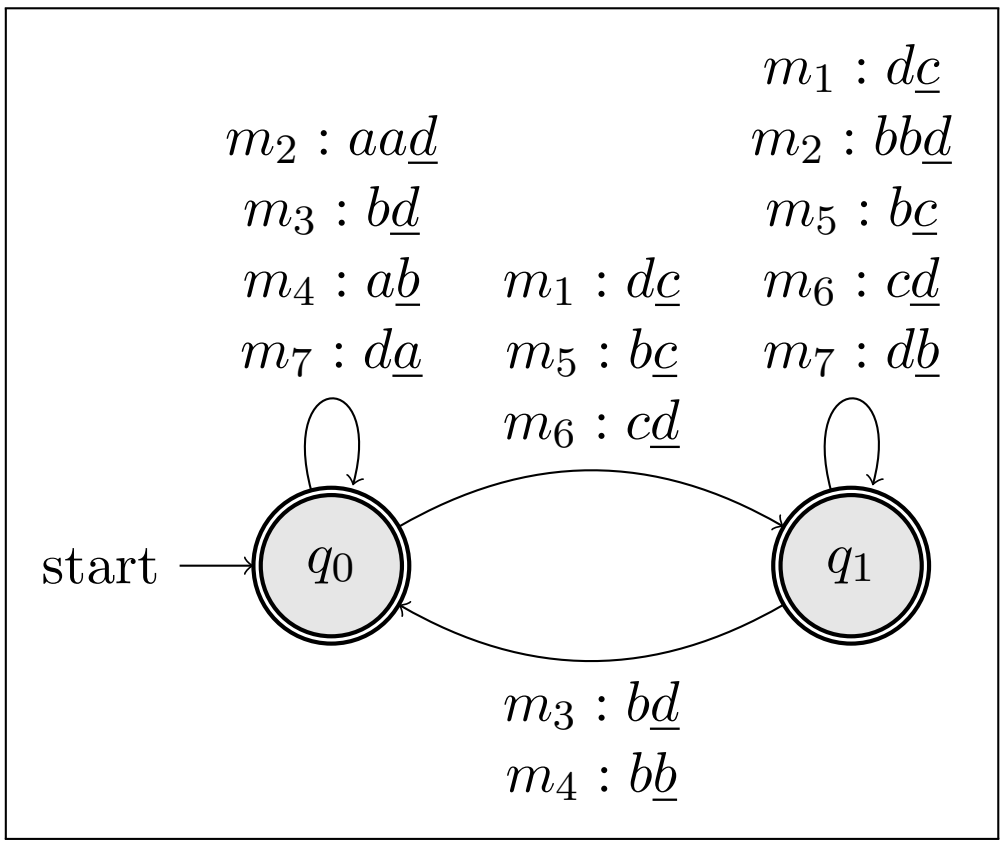
Data Consistent with $T_g \circ T_f$

We get sufficient data D consistent with $T_g \circ T_f$ and hand it to OSTIA, which gives us a transducer $T_h = T_{\hat{h}} = T_g \circ T_f$.



Get Output Suffixes

We find the output suffixes for each state:

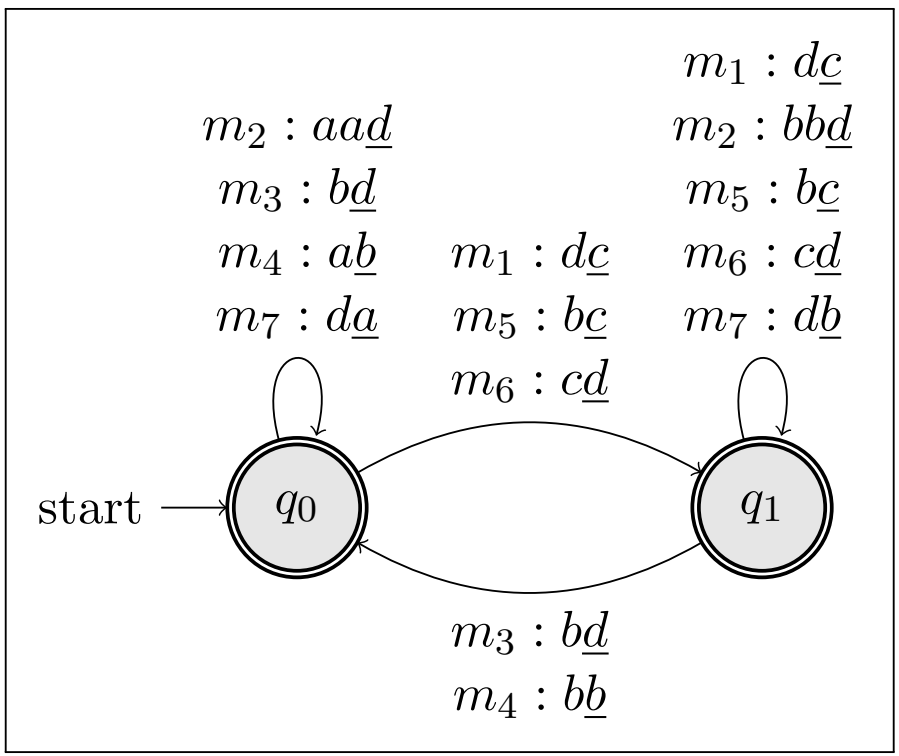


$\texttt{OS}(q_0) = \{\lambda, a, b, d\}$

$\texttt{OS}(q_1) = \{b, c, d\}$

Initialize T_g

Initialize T_g and create bijective mapping `corr` between T_g and T_h, based on cardinality of output suffixes with respect to each state.

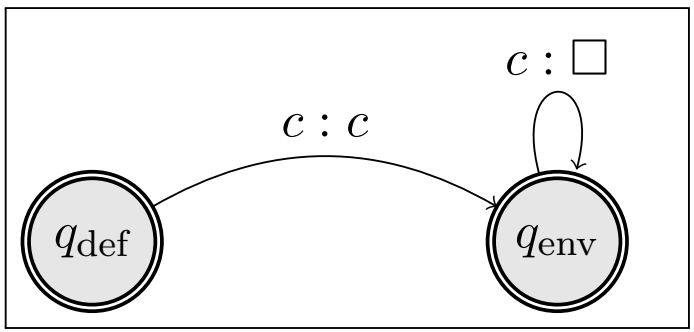




$\texttt{corr}(q_{\text{def}}) = q_0$
$\texttt{corr}(q_{\text{env}}) = q_1$

Find Environment

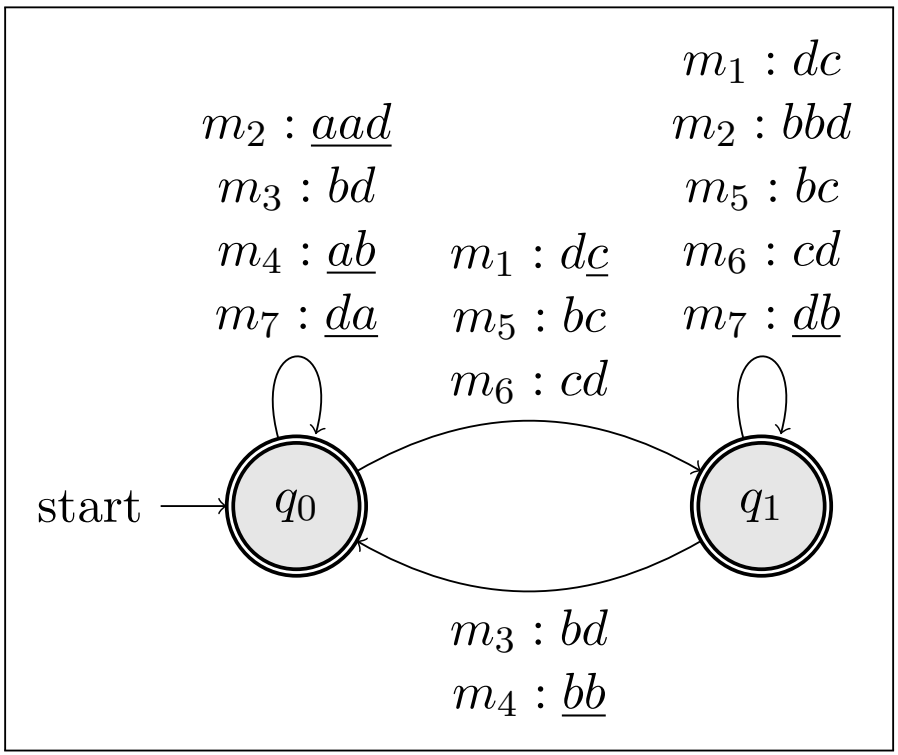
We find our the symbol that takes us to the environment by removing the intersection of the output suffixes from both states:

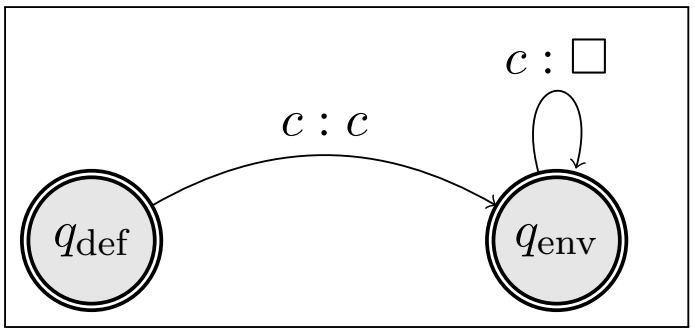


$$\texttt{OS}(q_0) = \{\lambda, a, b, d\}$$
$$\texttt{OS}(q_1) = \{b, c, d\}$$
$$\texttt{IS}(q_{\text{def}}) = \{\lambda, a, b, d\}$$
$$\texttt{IS}(q_{\textbf{env}}) = \{b, c, d\}$$
$$\texttt{IS}(q_{\text{env}}) - (\texttt{IS}(q_{\textbf{def}} \cap \texttt{IS}(q_{\textbf{env}}) = \{c\}$$

Finding Neutral Symbols

We find the morphemes have differing output on q_0 and q_1.





Consider the different outputs of m_7: da and db.

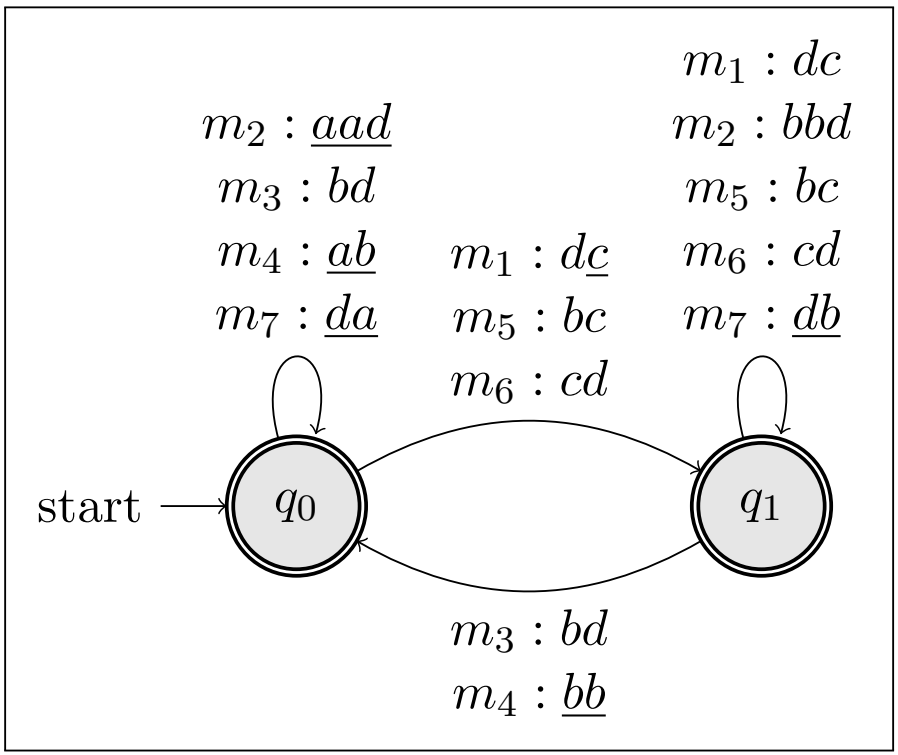
We know the change takes place coming out of q_{env}.

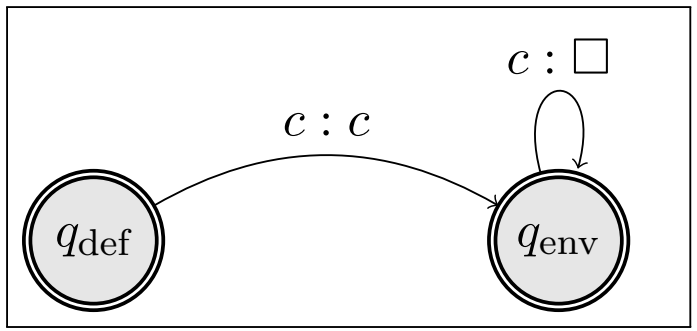
But d is not changing itself, so we must have stayed in q_{env}.

We infer $d \notin \tau$.

Finding Neutral Symbols

We find the morphemes have differing output on q_0 and q_1.

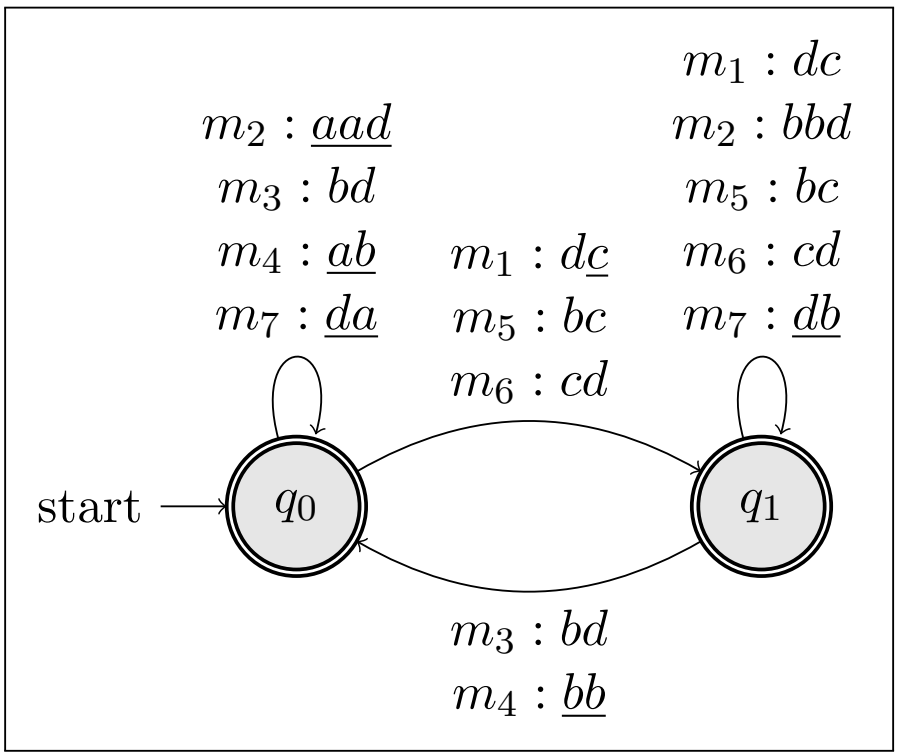


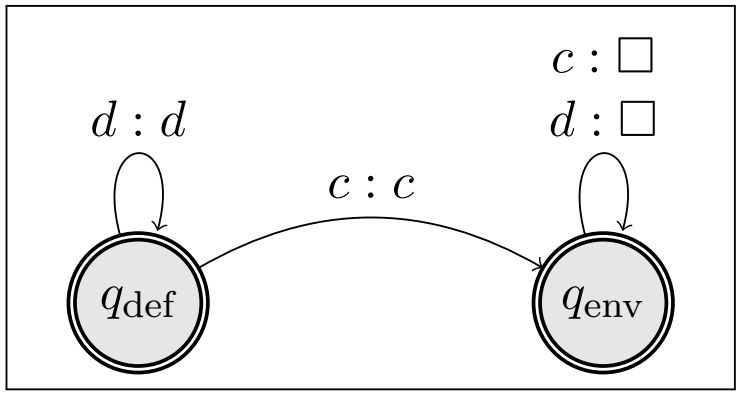


Consider the different outputs of m_7: da and db.
We know the change takes place coming out of q_{env}.
But d is not changing itself, so we must have stayed in q_{env}.
We infer $d \notin \tau$.

Finding Neutral Symbols Cont'd

But there's more!





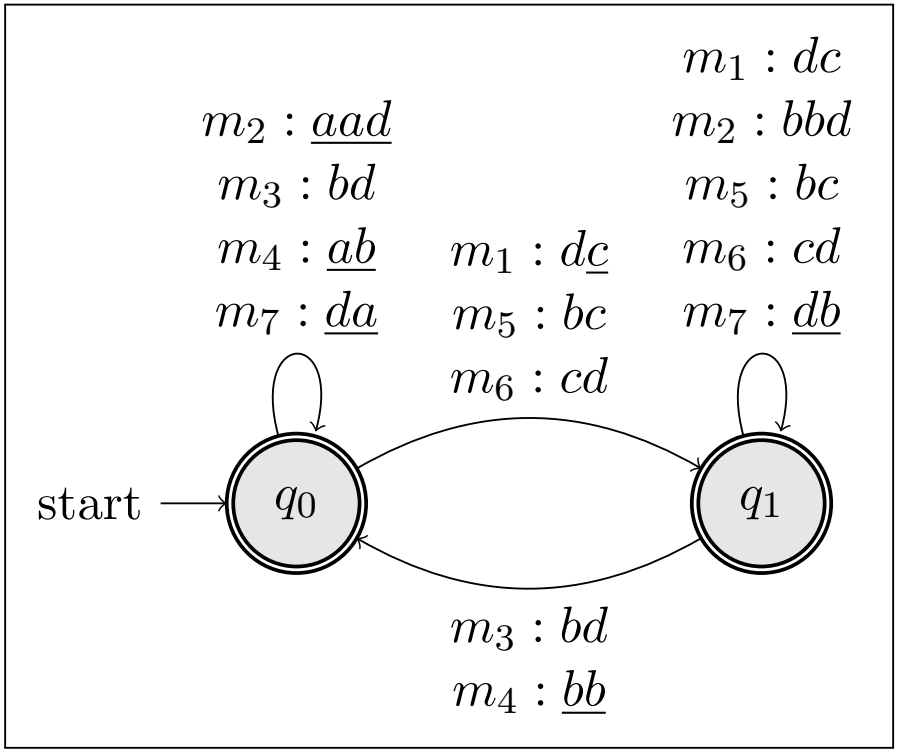
Consider the different outputs of m_2: *aad* and *bbd*.

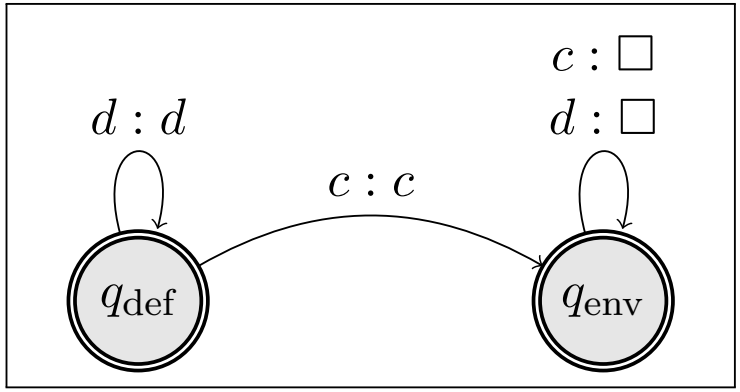
We know the change takes place coming out of q_{env}.

The initial symbol changes, but so does the string following it, so we must have stayed in q_{env}.

Finding Neutral Symbols Cont'd

But there's more!

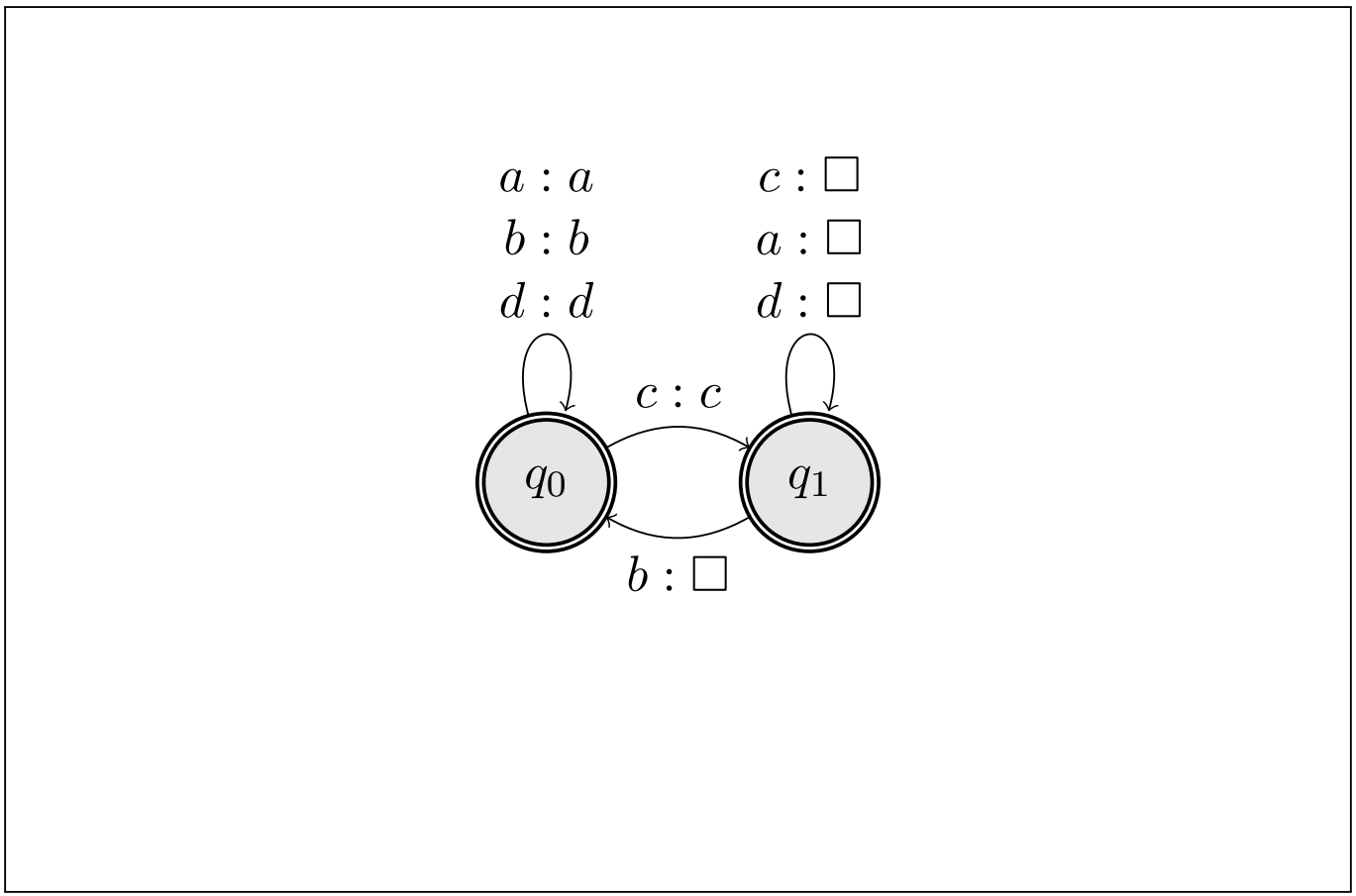




Consider the different outputs of m_2: *aad* and *bbd*.
We know the change takes place coming out of q_{env}.
The initial symbol changes, but so does the string following it, so we must have stayed in q_{env}.

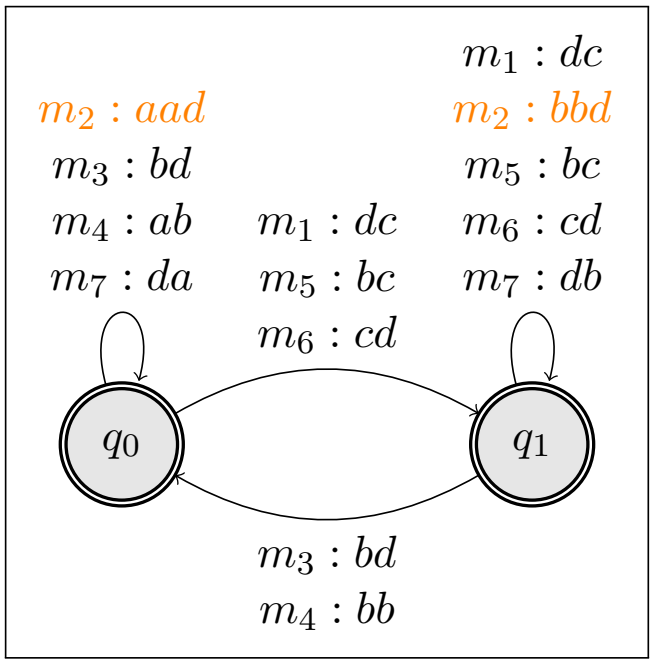
The Other Transitions

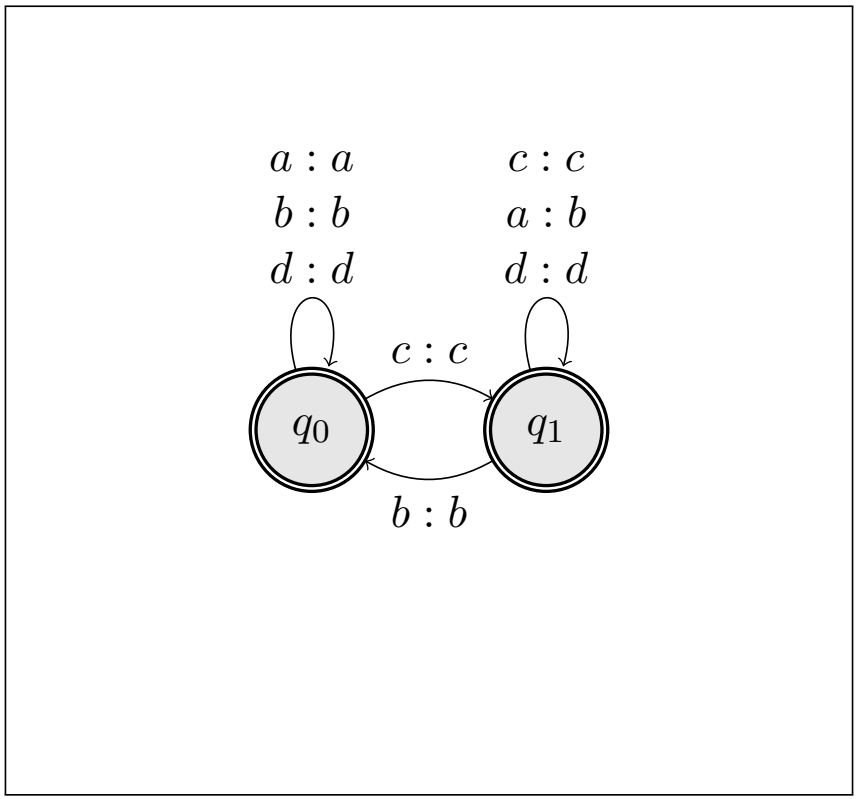
We assume everything else is an element of the tier.



Inference

Then we infer the rest exactly as in SI_2DLA.





Discussion

- We show that minimal modifications are required to expand the types of functions in the scope of the learner.
- The algorithm is largely the same, but we incorporate information about the relationship between (non-)tier symbols and properties of the SFST.
- Here we considered an iterative process, but we are interested to see how this would differ from other TISL functions such as vowel harmony.

Conclusion

- We can learn length preserving TISL functions composed with a homomorphism, which means we can learn most TISL functions in natural language phonology.
- It is really hard to learn

References

W. Hua and A. Jardine. Learning input strictly local functions from their composition. In *International Conference on Grammatical Inference*, pages 47–65. PMLR, 2021.

D. Lambert and J. Heinz. Algebraic reanalysis of phonological processes described as output-oriented. In *Proceedings of the Society for Computation in Linguistics 2024*, pages 129–138, 2024.

J. Oncina, P. García, and E. Vidal. Learning subsequential transducers for pattern recognition interpretation tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 15(5):448–458, 1993.